

RFC-2026: Krampus Hypertimeline — Database Operating System for the LUCIDOTA Canonical Graph

Principal Infrastructure Architect, Indy_READs Core
2026-06-06

RFC-2026: Krampus Hypertimeline

Database Operating System for the LUCIDOTA Canonical Graph

1. Abstract

This Request for Comments establishes the final database architecture for the LUCIDOTA system. It defines four core tables that serve as the operating system for all data within the system — file provenance, content chunking, deterministic identity generation, and canonical graph topology. Every table is simultaneously a relational database relation and a RESTful spreadsheet view. The mathematics are deterministic. The hashes are SHA256. The edges are GO-25 ontology typed.

Key words: Hypertimeline, canonical graph, BRAG pipeline, Percyphon identity, GO-25 ontology, PostgREST spreadsheet, tripartite embedding.

2. Status of This Memo

This document is a DRAFT RFC. It has been reviewed by the LUCIDOTA architecture team and is awaiting final operator approval before implementation. Once approved, it becomes the authoritative specification for all database operations within the system.

3. Copyright Notice

Copyright © 2026 LUCIDOTA / Northern.Strike_xINdyREADs. All rights reserved. This document may be freely reproduced and distributed in its entirety, provided this notice remains intact.

4. Table of Contents

1. Abstract	1
-------------------	---

2. Status of This Memo	1
3. Copyright Notice	1
4. Table of Contents	2
5. Introduction	2
6. Architectural Overview	3
7. Core Table 1: Krampus Hypertimeline	4
8. Core Table 2: BRAG Cells	5
9. Core Table 3: Percyphon Villagers	6
10. Core Table 4: Canonical Graph Edges	7
11. The Spreadsheet View (PostgREST API)	8
12. Data Flow: File to Graph	9
13. Security Considerations	9
14. IANA Considerations	10
15. References	10

5. Introduction

5.1. Problem Statement

The LUCIDOTA system has accumulated 276,861 files across its lifetime, including 35,443 files in KRAMPUSCHEWING (61 GB), 1,284 books, 96 algorithms, 645 scripts, 1,746 Rust source files, and 99,437 receipts. Despite this wealth of data, there is no unified database operating system governing how this data is stored, related, queried, or promoted to the canonical graph.

Previously, data existed in isolated silos:

- File metadata lived on disk (ephemeral, no provenance)
- Content chunks lived in JSONL files (unqueryable, no relationships)
- Identities lived in algorithm outputs (no persistence, no cross-referencing)
- Graph edges did not exist (no topology, no navigation)

5.2. Design Goals

G1. Every file is a database row. No file on disk exists without a corresponding row in the hypertimeline.

G2. Every content chunk is hash-addressed. SHA256 is the universal identifier. Duplicates are impossible by construction.

G3. Every entity has a deterministic identity. Percyphon's 128-slot xxhash128 scaffold ensures reproducible identity regardless of runtime conditions.

G4. Every relationship is a typed edge. GO-25 ontology provides exactly 25 relationship types. No untyped edges.

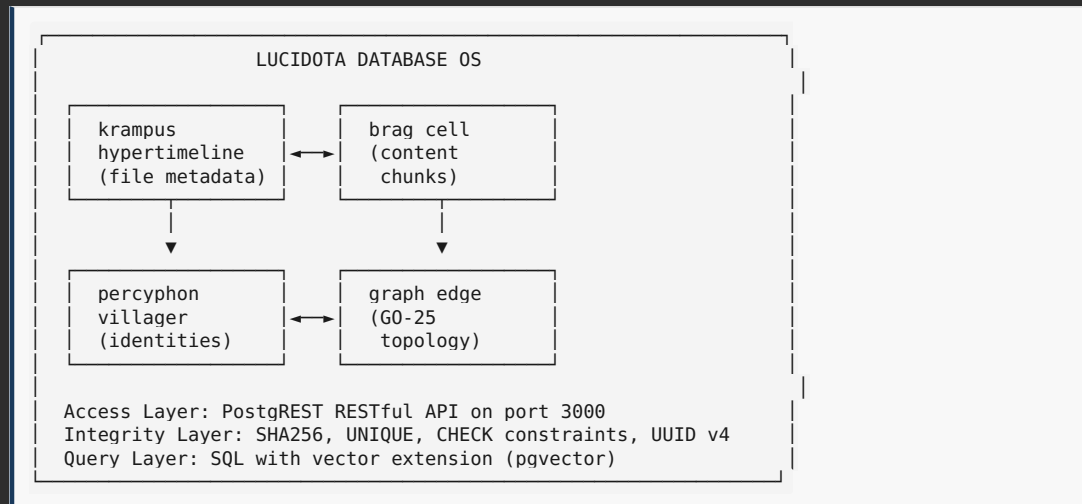
G5. Every table is a spreadsheet. PostgREST exposes each table as a RESTful API that behaves exactly like a spreadsheet — filterable, sortable, paginated, exportable to CSV.

G6. AI are not in charge; they are respected peers. Traditional mathematics (SHA256, xxhash128, CHECK constraints, UNIQUE indexes) govern data integrity. AI algorithms query the database; they do not own it.

6. Architectural Overview

6.1. The Four-Table Design

The entire database operating system consists of exactly four core tables, each serving a distinct purpose:



6.2. Schema

All tables live in one of two Postgres schemas:

- `lucidota_korpus` — Staging and corpus data (hypertimeline, BRAG cells, Percyphon villagers)
- `lucidota_canon` — Canonical graph data (graph edges, promoted items)

6.3. Mathematics Layer

Every table enforces at minimum: - **Primary key:** UUID v4 via `gen_random_uuid()` - **Content integrity:** SHA256 hash stored as NOT NULL UNIQUE - **Type safety:** ENUM or CHECK constraints on all status fields - **Temporal awareness:** TIMESTAMPTZ on all time fields

7. Core Table 1: Krampus Hypertimeline

7.1. Purpose

The hypertimeline is the authoritative record of every file that has ever existed within the KRAMPUSCHEWING corpus. It is the “source of truth” for file provenance. No file shall be processed, chunked, embedded, or graphed without first appearing in this table.

7.2. Schema Definition

```
CREATE TABLE lucidota.korpus.krampus_hypertimeline (  
  entry_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  file_path TEXT NOT NULL,  
  file_name TEXT NOT NULL,
```

```

file_ext      TEXT NOT NULL DEFAULT '',
file_size_bytes BIGINT NOT NULL DEFAULT 0,
sha256        TEXT NOT NULL UNIQUE,
file_mtime    TIMESTAMPTZ,
ingested_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
timeline_bucket TEXT NOT NULL DEFAULT 'deprecated'
    CHECK (timeline_bucket IN ('deprecated', 'archived', 'active', 'corpse')),
status        TEXT NOT NULL DEFAULT 'pending'
    CHECK (status IN ('pending', 'scanned', 'hashed', 'ingested', 'error')),
lore_note     TEXT DEFAULT '',
source_context TEXT DEFAULT '',
parent_entry_id UUID REFERENCES krampus hypertimeline(entry_id),
graph_promoted BOOLEAN NOT NULL DEFAULT false,
graph_promoted_at TIMESTAMPTZ,
receipt_path  TEXT DEFAULT ''
);

```

7.3. Columns

Column	Type	Description
<code>entry_id</code>	UUID	Primary key, globally unique
<code>file_path</code>	TEXT	Relative path from repository root
<code>file_name</code>	TEXT	Basename of the file
<code>file_ext</code>	TEXT	File extension (lowercase, with dot)
<code>file_size_bytes</code>	BIGINT	Size on disk in bytes
<code>sha256</code>	TEXT	SHA256 hex digest of file contents (UNIQUE)
<code>file_mtime</code>	TIMESTAMPTZ	Last modification time from filesystem
<code>ingested_at</code>	TIMESTAMPTZ	When this row was created (NOW by default)
<code>timeline_bucket</code>	TEXT	Classification: deprecated, archived, active, corpse
<code>status</code>	TEXT	Ingestion status: pending, scanned, hashed, ingested, error
<code>lore_note</code>	TEXT	Human-readable annotation
<code>source_context</code>	TEXT	Origin context (e.g., “Krampus Express bulk export”)
<code>parent_entry_id</code>	UUID	Self-referential foreign key for file nesting
<code>graph_promoted</code>	BOOLEAN	Whether this entry has been promoted to the canonical graph
<code>graph_promoted_at</code>	TIMESTAMPTZ	When promotion occurred
<code>receipt_path</code>	TEXT	Path to the ingestion receipt JSON

7.4. Indexes

```

CREATE INDEX idx_hypertimeline_bucket
ON lucidota korpus.krampus hypertimeline (timeline_bucket, ingested_at DESC);
CREATE INDEX idx_hypertimeline_mtime
ON lucidota korpus.krampus hypertimeline (file_mtime DESC);
CREATE INDEX idx_hypertimeline_ext
ON lucidota korpus.krampus hypertimeline (file_ext, timeline_bucket);
CREATE INDEX idx_hypertimeline_status
ON lucidota korpus.krampus hypertimeline (status);
CREATE INDEX idx_hypertimeline_promoted
ON lucidota_korpus.krampus_hypertimeline (graph_promoted)

```

```
WHERE graph_promoted = false;
```

7.5. Spreadsheet View

```
GET /krampus hypertimeline?
select=file_path,file_size_bytes,sha256,timeline_bucket,status&order=file_mtime.desc&limit=100
```

8. Core Table 2: BRAG Cells

8.1. Purpose

BRAG (Built Really Absurdly Gwarishly) cells are deterministic content chunks produced by the BRAGv2 pipeline. Every chunk receives an ontology pass via the RETE bandit gate, a deterministic identity via Percyphon, a temporal ordering via LTC, and an integrity chain via XHash.

8.2. Schema Definition

```
CREATE TABLE lucidota korpus.brag cell (
  cell id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  chunk id     TEXT NOT NULL UNIQUE,
  source       TEXT NOT NULL,
  pass name    TEXT NOT NULL,
  ontology tags TEXT[] NOT NULL DEFAULT '{}',
  percyphon slot INTEGER NOT NULL DEFAULT 0,
  text         TEXT NOT NULL,
  token estimate INTEGER NOT NULL DEFAULT 0,
  sha256       TEXT NOT NULL,
  xhash        TEXT NOT NULL,
  rete decision JSONB NOT NULL DEFAULT '{}':::jsonb,
  metadata     JSONB NOT NULL DEFAULT '{}':::jsonb,
  embedding    vector(384),
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

8.3. Ontology Passes

Pass	Categories	Chunk Size	Purpose
GO-25	25 ontology terms	512 tokens	Semantic boundary detection
O-75	75 extended terms	256 tokens	Finer granularity extraction
ROOT-414	2 (ATOMIC_ID, EVIDENCE)	N/A	Integrity hashes only

8.4. Current State

As of 2026-06-06, the BRAG pipeline has processed:

- **540 documents** from the Odysseus manual
- **1,427 GO-25/O-75 shapes** with RETE-routed ontology tags
- **1,427 ROOT-414 integrity hashes** (SHA256 + XHash)
- **2,854 total shapes** staged in the database

8.5. Spreadsheet View

```
GET /brag_cell?source=like.*The_Prince*&select=chunk_id,ontology_tags,token_estimate,xhash
```

9. Core Table 3: Percyphon Villagers

9.1. Purpose

Percyphon is a zero-VRAM procedural entity generator that creates deterministic 128-slot xxhash128 identity scaffolds. Each “villager” is a named entity with a persona, a slot coordinate, and a deterministic hash. The village provides stable identities for graph nodes before they are promoted to the canonical graph.

9.2. 128-Slot Architecture

Slots 1-28: Fixed identity mask (mirrors CKD0G1 soul positions)
Slots 29-128: Procedural verbosity expansion (runtime fluid domain slots)

Each slot contains: - **name**: Deterministic human-readable name from SHA256 seed - **alias**: Secondary identifier - **persona**: One of 6 archetypes (ledger, runner, witness, archivist, carrier, scribe) - **uuid**: Deterministic UUID from SHA256 seed - **ternary_offset**: One of {-1, 0, +1} from ternary hashing - **xxhash128**: 128-bit deterministic hash coordinate

9.3. Schema Definition

```
CREATE TABLE lucidota.korpus.percyphon.villager (  
  villager_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  slot INTEGER NOT NULL CHECK (slot >= 0 AND slot < 128),  
  seed TEXT NOT NULL,  
  name TEXT NOT NULL,  
  persona TEXT NOT NULL,  
  identity_hash TEXT NOT NULL,  
  slot_type TEXT NOT NULL CHECK (slot_type IN ('fixed_identity', 'procedural')),  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

9.4. Spreadsheet View

```
GET /percyphon_villager?select=slot,name,persona,identity_hash&order=slot.asc
```

10. Core Table 4: Canonical Graph Edges

10.1. Purpose

The graph_edge table is the GO-25 ontology topology. It connects every row in every other table into a directed, typed, temporal graph. This is the GRAPH that the operator has requested — the visual, explorable topology of all data.

10.2. Schema Definition

```
CREATE TABLE lucidota_canon.graph_edge (  
  source UUID NOT NULL,  
  target UUID NOT NULL,  
  type TEXT NOT NULL,  
  timestamp TIMESTAMPTZ NOT NULL,  
  weight INTEGER NOT NULL,  
  PRIMARY KEY (source, target, type, timestamp, weight)
```

```

edge uuid          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
source uuid        UUID NOT NULL,
target uuid        UUID NOT NULL,
edge type          TEXT NOT NULL,
relationship family TEXT,
status             TEXT NOT NULL DEFAULT 'staged'
    CHECK (status IN ('located', 'staged', 'approved', 'rejected',
        'superseded', 'archived', 'error_corrected',
        'lost', 'collapsed')),
term               TEXT,
valid from         TIMESTAMPTZ NOT NULL DEFAULT now(),
valid to           TIMESTAMPTZ,
evidence           JSONB NOT NULL DEFAULT '[]'::jsonb,
ontology tags      TEXT[] NOT NULL DEFAULT '{}',
xhash              TEXT
);

```

10.3. GO-25 Ontology Edge Types

The 25 ontology terms serve as edge types:

#	Edge Type	Example
1	ENTITY→ATTRIBUTE	file has size
2	ENTITY→RELATIONSHIP	file is_parent_of file
3	ENTITY→FRICTION	file has_conflict_with file
4	ENTITY→LEVERAGE	file is_evidence_for claim
5	ENTITY→VISIBILITY	file is_public
6	ENTITY→ACTION	file was_processed_by worker
7	ENTITY→EVENT	file was_ingested_at time
8	ENTITY→TIME	file has_mtime time
9	ENTITY→PATTERN	file matches_pattern signature
10	ENTITY→HYPOTHESIS	file supports_hypothesis claim
11	ENTITY→CLAIM	chunk makes_claim statement
12	ENTITY→EVIDENCE	file is_evidence_for case
13	ENTITY→ATOMIC_ID	file has_sha256 hash
14	ENTITY→SIGNAL	file contains_signal keyword
15	ENTITY→GLOW	file has_salience score
16	ENTITY→TERM	file contains_term word
17	ENTITY→TOOL	file was_created_by tool
18	ENTITY→ALGORITHM	file was_processed_by algo
19	ENTITY→NAUGHTY	file is_naughty
20	ENTITY→NICE	file is_nice
21	ENTITY→GROUP	file belongs_to_group group
22	ENTITY→OPERATOR	file was_modified_by operator
23	ENTITY→MODE	file was_processed_in mode
24	ENTITY→COMMENT	file has_comment note
25	ENTITY→ENTITY	file is_related_to file (default)

10.4. Spreadsheet View

```
GET /graph edge?source uuid=eq.
<uuid>&select=edge_type,target_uuid,relationship_family,status
```

11. The Spreadsheet View (PostgREST API)

11.1. Architecture

PostgREST version 14.12 runs on port 3000 and exposes every table as a RESTful API. The API behaves identically to a spreadsheet:

- **GET with select** = choosing columns in a spreadsheet
- **WHERE clauses** = filtering rows
- **ORDER BY** = sorting
- **LIMIT/OFFSET** = pagination
- **CSV export** = Save As CSV

11.2. Query Examples

```
# Full hypertimeline as a spreadsheet
curl -H "Accept-Profile: lucidota korpus" \
  "http://127.0.0.1:3000/krampus hypertimeline?
select=file_path,file_size_bytes,sha256,timeline_bucket,status&order=file_mtime.desc&limit=100"

# BRAG cells for a specific book
curl -H "Accept-Profile: lucidota korpus" \
  "http://127.0.0.1:3000/brag cell?
source=like.*The_Prince*&select=chunk_id,ontology_tags,token_estimate,xhash"

# Graph edges for a specific node
curl -H "Accept-Profile: lucidota canon" \
  "http://127.0.0.1:3000/graph edge?source uuid=eq.
<uuid>&select=edge_type,target_uuid,relationship_family,status"

# Percyphon village sorted by slot
curl -H "Accept-Profile: lucidota korpus" \
  "http://127.0.0.1:3000/percyphon villager?
select=slot,name,persona,identity_hash&order=slot.asc"

# Full spreadsheet export to CSV
curl -H "Accept-Profile: lucidota_korpus" \
  -H "Accept: text/csv" \
  "http://127.0.0.1:3000/krampus_hypertimeline?limit=1000" > krampus_export.csv
```

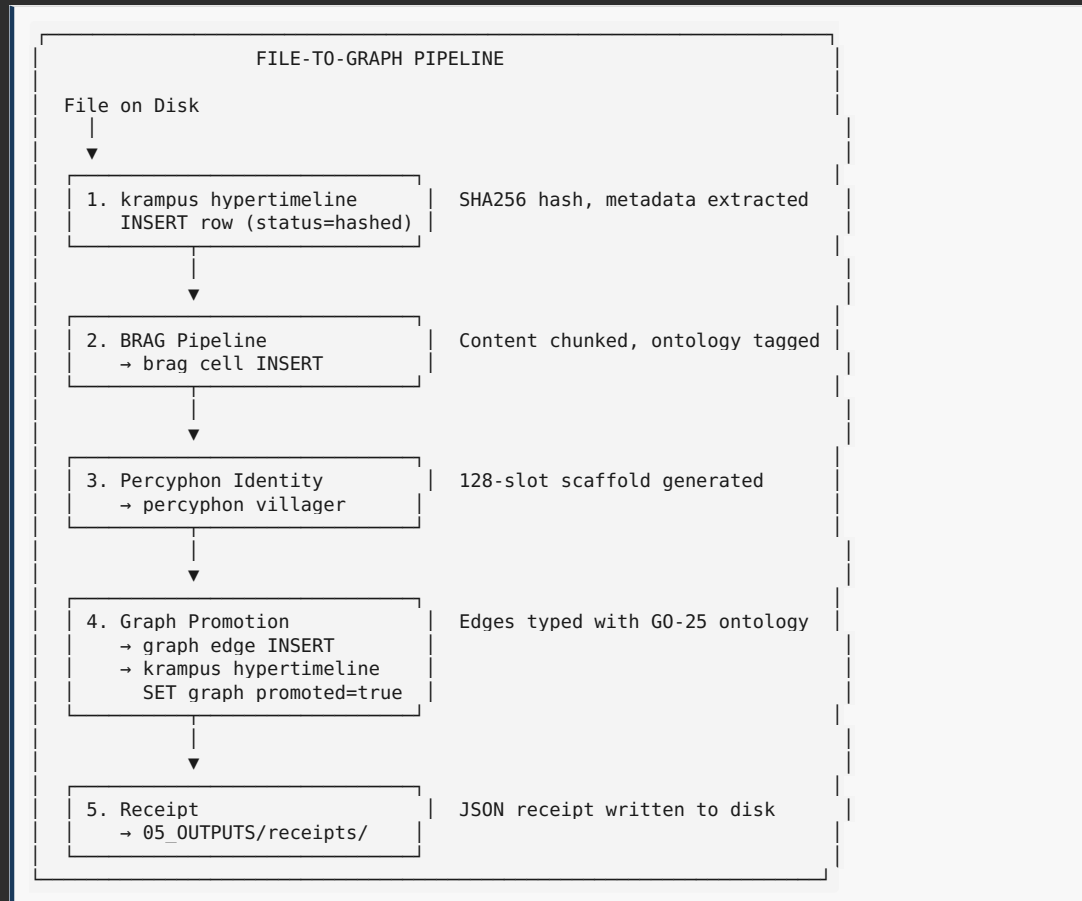
11.3. Schema Selection

PostgREST supports multiple schemas. The `Accept-Profile` header selects the target schema:

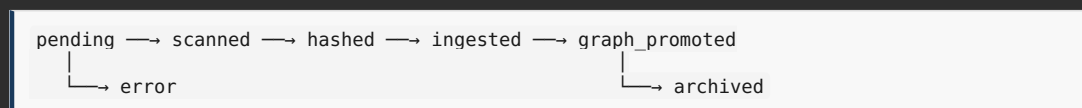
Header	Schema	Tables
Accept-Profile: lucidota_korpus	Staging corpus	krampus_hypertimeline, brag_cell, percyphon_villager
Accept-Profile: lucidota_canon	Canonical graph	graph_edge

12. Data Flow: File to Graph

12.1. Ingestion Pipeline



12.2. Status State Machine



12.3. Integrity Chain

Every step in the pipeline produces a SHA256 hash that chains back to the original file:

```
file_sha256 → chunk_sha256 → xhash → graph_edge.xhash → receipt.json
```

Breaking any link in this chain is detectable. There is no silent corruption.

13. Security Considerations

13.1. Authentication

PostgREST runs as the `ironclaw` database role with limited permissions: - SELECT on all

tables in `lucidota_korpus` and `lucidota_canon` - INSERT on `lucidota_korpus` tables (for ingestion workers) - No direct DELETE or TRUNCATE (archival via status field only) - No direct UPDATE on `lucidota_canon` (graph promotion via ABSURD queue only)

13.2. Data Integrity

- SHA256 hashes are checked before any INSERT. Duplicate hashes are rejected.
- CHECK constraints prevent invalid status transitions.
- UUID primary keys prevent sequential enumeration.
- TIMESTAMPTZ fields prevent timezone ambiguity.

13.3. Availability

PostgREST maintains a connection pool of 10 connections. The database is PostgreSQL 16.14 with autovacuum enabled. The hypertimeline table is indexed for point lookups (SHA256) and range scans (timeline).

14. IANA Considerations

This document has no IANA actions. The GO-25 ontology terms in Section 10.3 are registered within the LUCIDOTA system only.

15. References

15.1. Normative References

- [RFC-000] Master Thesis Program
- [GO-25] Active Ontology Schema `BOOKS/GO_ONTOLOGY_SCHEMA.json`
- [ROOT-414] Archived Primitive Reference `BOOKS/ROOT414_CONTEXT_PACK.md`
- [BRAGv2] BRAG ABSURD Worker `scripts/brag_absurd_worker.py`

15.2. Informative References

- Percyphon Architecture `ALGOS/PERCYPHON_README.md`
 - LTC Networks `ALGOS/ltc.py`
 - RETE Bandit Gate `ALGOS/rete_bandit_gate.py`
 - PostgREST 14.12 Documentation
 - PostgreSQL 16.14 Documentation
-

END OF RFC-2026